MLP from Scratch na Base Íris: Depuração da Implementação, Classificação das Três Espécies e Análise de Convergência e Overfitting

Gustavo Gabriel Ripka
GRR20203935

Arthur Barreto Godoi
GRR20224377

Departamento de Informática
Universidade Federal do Paraná – UFPR
Curitiba, Brasil

***Resumo*—Este trabalho estuda, depura e experimenta uma implementação de Perceptron Multicamadas (MLP) *from scratch* — sem frameworks de aprendizado profundo — na tarefa de classificar as três espécies da base Íris. A análise do código original revelou cinco defeitos relevantes, dos quais o mais grave era o uso de `random.shuffle` sobre um `ndarray` bidimensional do NumPy: as trocas de linhas viram duplicações por *aliasing* de *views*, perdendo amostras e quase eliminando a classe *virginica* do treino — essa corrupção, e não o algoritmo de retropropagação (que estava correto), era o que impedia classificar bem as três classes. Com as correções e divisão estratificada 80/20, a configuração original já atinge 96,7% de acurácia de teste. Experimentos sistemáticos (ativação $\times$ neurônios ocultos $\times$ learning rate, 3 sementes) mostraram que a normalização z-score exige recalibrar o learning rate (0,005 $\rightarrow$ 0,01–0,05) para não degradar o desempenho, que a `sigmoid` atinge piso de erro menor que a `tanh` com alvos one-hot, e que o ReLU morre precocemente com a inicialização em U$(-1,1)$ e bias -1. A melhor configuração (`tanh`, 5 neurônios ocultos, lr 0,01) alcançou 97,8% $\pm$ 1,9 de acurácia média de teste e, com 1500 épocas, 100% no conjunto de teste. O sobreajuste foi medido pelo intervalo treino–teste: modesto na Íris, cresce com o tamanho da camada oculta.**

***Index Terms*—perceptron multicamadas, retropropagação, Íris, depuração, convergência, overfitting.**

I. Introdução

O Perceptron Multicamadas (MLP) treinado por retropropagação do gradiente é a arquitetura neural mais elementar capaz de separar classes não linearmente [2]. Implementá-lo *from scratch* — sem `scikit-learn`, PyTorch ou TensorFlow — expõe todos os detalhes que frameworks escondem: inicialização de pesos, codificação de alvos, atualização de biases, embaralhamento dos dados e cálculo do erro. É também um exercício propenso a bugs discretos, cujos sintomas (acurácia baixa, erro que não desce) raramente apontam diretamente para a causa.

Este trabalho parte da implementação de MLP apresentada em aula (adaptada do *notebook* de Vitor Gama Lemos [3]) e tem três objetivos, conforme o enunciado: (i) estudar a implementação e ajudá-la a depurar; (ii) descobrir como classificar bem as três espécies da base Íris [1] — 150 amostras, 4 atributos morfológicos de sépalas e pétalas, 3 classes equilibradas; e (iii) analisar convergência, erros por classe e sobreajuste.

A contribuição central é o diagnóstico: o código original não conseguia classificar as três classes por causas *externas* ao algoritmo de aprendizado — dados corrompidos no embaralhamento e avaliação sem função de ativação. Identificados e corrigidos esses defeitos, o mesmo backpropagation passa a atingir 100% de acurácia de teste na melhor configuração. Relatamos também dois fenômenos instrutivos dos experimentos: a interação entre normalização de atributos e learning rate, e a incompatibilidade entre ReLU e o esquema de inicialização adotado.

II. A implementação e seus defeitos

A rede possui uma camada oculta e é treinada amostra a amostra (estocástico). Dada a entrada $\mathbf{x} \in \mathbb{R}^4$, a propagação direta é

$$\mathbf{h} = \phi\Big(\mathbf{x}\,\mathbf{W}^{(1)} + \mathbf{b}^{(1)}\Big), \tag{1}$$

$$\mathbf{o} = \phi\Big(\mathbf{h}\,\mathbf{W}^{(2)} + \mathbf{b}^{(2)}\Big), \tag{2}$$

com ϕ a função de ativação (`sigmoid`, `tanh` ou ReLU), alvos codificados em one-hot $\{0,1\}$ e erro quadrático médio

$$E = \frac{1}{2n}\sum_{i=1}^{n}\|\mathbf{t}_i - \mathbf{o}_i\|^2. \tag{3}$$

Os pesos são inicializados uniformemente em $[-1,+1]$ e os biases iniciam em -1, como no original. A retropropagação segue o esquema clássico: o delta da saída é $\boldsymbol{\delta}^{(2)} = (\mathbf{o}-\mathbf{t}) \odot \phi'(\mathbf{o})$, o delta oculto é retropropagado pelos pesos sinápticos, $\boldsymbol{\delta}^{(1)} = (\mathbf{W}^{(2)}\boldsymbol{\delta}^{(2)}) \odot \phi'(\mathbf{h})$, e cada peso/bias desce pelo gradiente com taxa de aprendizado η. Este núcleo estava **correto**; os problemas estavam à sua volta:

1) **`predict()` sem função de ativação.** A propagação direta usada na avaliação não aplicava ϕ — a rede era avaliada como um modelo *linear*. O treino aprendia não-linearidades que a avaliação ignorava, destruindo a acurácia mesmo com a rede bem treinada.
2) **Erro acumulado entre épocas.** O acumulador de erro nunca era zerado ao início de cada época, de modo que

o MSE exibido *crescia monotonicamente* com o número de épocas — a rede parecia divergir mesmo quando melhorava.

3) **Fator $0{,}05$ arbitrário no erro e registro a cada 50 épocas.** A curva de convergência ficava distorcida em escala e sem resolução temporal adequada.
4) **Typo `iasHiddenValue` e ausência de semente.** O parâmetro de bias oculto quebrava silenciosamente a configuração por dicionário de parâmetros, e sem semente própria os pesos não eram reprodutíveis.
5) **`random.shuffle` sobre `ndarray` 2D — o bug mais grave.** O módulo `random` do Python embaralha por trocas `x[i], x[j] = x[j], x[i]`. Em um `ndarray` 2D, os lados criam *views* que se sobrescrevem durante a própria atribuição: a operação degenera em `x[i] = x[j]` duas vezes, *duplicando* linhas e *perdendo* outras. Demonstramos o efeito diretamente: após o embaralhamento ingênuo, as 150 amostras deixam de ser 50/50/50 e a classe *virginica* quase desaparece. A rede era então treinada e avaliada num conjunto corrompido — era isso, e não o algoritmo, que impedia classificar bem as três classes. A correção é embaralhar *índices* (ou delegar ao `train_test_split`).

A ausência de normalização de atributos **não** é um defeito — mas, como mostram os experimentos, interage fortemente com o learning rate. A versão corrigida registra o MSE de *todas* as épocas, mantém histórico de acurácia treino/teste por época (para análise de sobreajuste) e aceita semente explícita para reprodutibilidade.

III. Metodologia experimental

A. Preparação dos dados

Após a correção, o conjunto é embaralhado por índices e dividido em 80/20 com estratificação por classe (`random_state`=11): 120 amostras de treino e 30 de teste, 40/40/40 e 10/10/10 por classe. A normalização z-score, quando adotada, usa média e desvio *apenas do treino*, reaplicados ao teste — sem vazamento de informação.

B. Protocolo

Todos os experimentos usam a mesma classe corrigida, variando apenas hiperparâmetros. A grade sistemática combina 3 ativações $\times$ 2 tamanhos de camada oculta (5, 10) $\times$ 2 learning rates (0,01; 0,05), com **3 sementes** por célula e 1000 épocas, sempre sobre atributos normalizados; reportamos média e desvio-padrão da acurácia de teste para não concluir de uma única sorte de inicialização. O sobreajuste é analisado de duas formas: curvas de acurácia treino/teste por época e o intervalo (gap) treino$-$teste em função do tamanho da camada oculta (2, 5, 10, 20; `tanh`, lr 0,05, 1200 épocas, 3 sementes).

IV. Resultados

A. Antes e depois da depuração

Com o conjunto íntegro e os demais bugs corrigidos, a configuração original (`sigmoid`, 5 neurônios ocultos, lr 0,005, 3000 épocas, atributos crus) passa a exibir curva de MSE *decrescente* — de 0,44 até 0,026 — e acurácias de 97,5% (treino) e 96,7% (teste). Ou seja: o backpropagation original já era capaz disso; os defeitos de avaliação e de dados é que o escondiam.

Tabela I
GRADE DE EXPERIMENTOS (Z-SCORE, 1000 ÉPOCAS, MÉDIA DE 3 SEMENTES; ORDENADA PELA ACURÁCIA DE TESTE)

Ativação	h	lr	Acc. treino	Acc. teste ($\pm$ dp)
tanh	5	0,01	0,978	**0,978** $\pm$ 0,019
sigmoid	5	0,05	0,975	**0,978** $\pm$ 0,019
sigmoid	10	0,05	0,986	0,967 $\pm$ 0,000
tanh	10	0,01	0,986	0,967 $\pm$ 0,000
tanh	10	0,05	0,975	0,956 $\pm$ 0,019
tanh	5	0,05	0,975	0,922 $\pm$ 0,019
sigmoid	10	0,01	0,939	0,900 $\pm$ 0,000
sigmoid	5	0,01	0,922	0,900 $\pm$ 0,000
ReLU	10	0,01	0,750	0,744 $\pm$ 0,135
ReLU	5	0,01	0,442	0,444 $\pm$ 0,192
ReLU	5	0,05	0,350	0,344 $\pm$ 0,019
ReLU	10	0,05	0,336	0,333 $\pm$ 0,000

B. Normalização $\times$ learning rate

Repetindo a configuração original com z-score *sem* ajustar o learning rate, o resultado *piora*: MSE final 0,072 (contra 0,026) e acurácia de teste de 90,0% (contra 96,7%). O aparente paradoxo tem explicação: a normalização reduz a escala das entradas e, com ela, a magnitude dos gradientes; o lr 0,005, calibrado *de facto* para entradas grandes, torna-se conservador demais. A grade da Tabela I confirma que basta **recalibrar o lr** (0,01–0,05) para a rede normalizada igualar e superar a versão crua.

C. Convergência por função de ativação

Nas curvas de MSE (h=5, lr 0,05, mesma semente), a `sigmoid` atinge piso de erro inferior ao da `tanh` (MSE final 0,031 contra 0,094) — com alvos one-hot $\{0, 1\}$, a `tanh`, cujo alcance é $(-1, 1)$, não consegue representar exatamente 0 e 1, ainda que classifique corretamente pela margem. O ReLU, por sua vez, fica preso em MSE $\approx$ 0,5 e acurácia de 1/3 — *morte precoce de neurônios*: com pesos U($-$1,1) e bias -1, boa parte das pré-ativações é negativa, o gradiente do neurônio morto é zero e ele nunca se recupera. A lição é que ativação, inicialização e bias precisam ser compatíveis entre si.

D. Erros por classe e classificação perfeita

Treinando a melhor configuração da grade (`tanh`, h=5, lr 0,01) por 1500 épocas (semente 0), a rede atinge **100% de acurácia no conjunto de teste** (98,3% no treino), com matriz de confusão perfeita — as três classes *setosa*, *versicolour* e *virginica* todas em 100%. A análise por classe confirma a estrutura esperada da base: a *setosa* é linearmente separável das demais e nunca é confundida; os erros residuais das configurações mais fracas (ex.: `sigmoid` lr 0,005 normalizada, 90% de teste) concentram-se exclusivamente na fronteira *versicolour* $\times$ *virginica*, as duas espécies com sobreposição morfológica.

Tabela II
SOBREAJUSTE × TAMANHO DA CAMADA OCULTA (TANH, LR 0,05, 1200 ÉPOCAS, MÉDIA DE 3 SEMENTES)

h	Acc. treino	Acc. teste	Gap (p.p.)
2	0,958	0,967	$-0{,}8$
5	0,975	0,922	$+5{,}3$
10	0,978	0,956	$+2{,}2$
20	0,986	0,944	$+4{,}2$

E. Sobreajuste

A Tabela II resume o intervalo treino–teste pelo tamanho da camada oculta. O gap cresce com a capacidade — de $-0{,}8$ p.p. com h=2 até $+4$ a $+5$ p.p. com h=5/20 — mas permanece modesto na Íris: conjunto pequeno e bem separável após as correções. Nas curvas de acurácia por época (h=5 contra h=20), a curva de teste estaciona enquanto a de treino continua subindo, e o comportamento dominante do excesso de capacidade é a *oscilação* da acurácia de teste, mais do que queda acentuada.

V. DISCUSSÃO

O resultado mais instrutivo do trabalho é negativo, no bom sentido: *nenhuma* melhoria de hiperparâmetros era capaz de classificar as três classes enquanto os dados estivessem corrompidos e a avaliação, linear. O diagnóstico sistematizou-se em verificar cada elo da cadeia — dados → treino → avaliação — antes de culpar o algoritmo. A demonstração do defeito do `random.shuffle` é reproduzível em duas linhas (`np.bincount` antes/depois) e vale como advertência geral sobre operações de atribuição com *views* do NumPy.

A interação normalização × learning rate ilustra que hiperparâmetros não são independentes: "normalizar sempre ajuda" só vale se o lr for recalibrado junto. Já o fracasso do ReLU mostra que a arquitetura também é um sistema: ativação, inicialização e bias formam um trio que precisa ser coerente. Com pesos U(−1,1) e bias -1 — herdados do original —, `sigmoid` e `tanh` funcionam, e o ReLU não; uma inicialização tipo He com bias positivo resolveria, mas alteraria o espírito de preservar a implementação original.

Por fim, o desvio-padrão entre sementes (até $\pm$1,9 p.p. nas melhores células, $\pm$19 no ReLU instável) reforça que conclusões sobre redes pequenas em bases pequenas exigem múltiplas execuções: qualquer célula isolada da grade conta uma história diferente da média.

VI. CONCLUSÃO

Estudamos e depuramos uma implementação de MLP *from scratch* na base Íris, corrigindo cinco defeitos — o mais grave, o embaralhamento que corrompia o conjunto de dados e quase eliminava a classe *virginica*. O algoritmo de retropropagação original estava correto: com dados íntegros e avaliação não linear, a configuração original já atinge 96,7% de teste; a melhor da grade (`tanh`, 5 neurônios, lr 0,01, z-score) chega a 97,8% $\pm$ 1,9 e, com 1500 épocas, a **100% nas três classes**. A análise de convergência mostrou pisos de erro distintos por ativação (compatibilidade com alvos one-hot e com a inicialização), e a de sobreajuste, um gap treino–teste modesto que cresce com a capacidade. Como trabalho futuro, restam inicialização adequada ao ReLU, taxa adaptativa e validação cruzada k-fold para estimar variância com mais amostras.

REPOSITÓRIO

O *notebook* e os resultados estão disponíveis em: https://github.com/gg-gustavo/mlp-scratch-iris

REFERÊNCIAS

[1] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.

[2] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2009.

[3] V. G. Lemos, "Multilayer perceptron from scratch," Kaggle, 2024. [Online]. Disponível em: kaggle.com/code/vitorgamalemos/multilayer-perceptron-from-scratch